

# Introduction: the Machine Learning Landscape

# What is ML?

“A field of study that gives computers the ability to learn without being explicitly programmed.” – Arthur Samuel, 1959

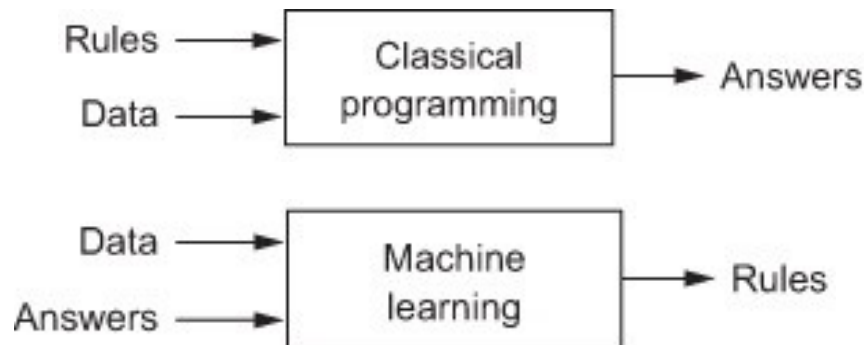
“A computer program is said to learn from experience  $E$  with respect to some task  $T$  and some performance measure  $P$ , if its performance on  $T$ , as measured by  $P$ , improves with experience  $E$ .” – Tom Mitchell, 1997

ML is a prominent sub-field in AI, “the new electricity.” – Andrew Ng

(<https://www.youtube.com/watch?v=21EiKfQYZXc>)

# What is ML?

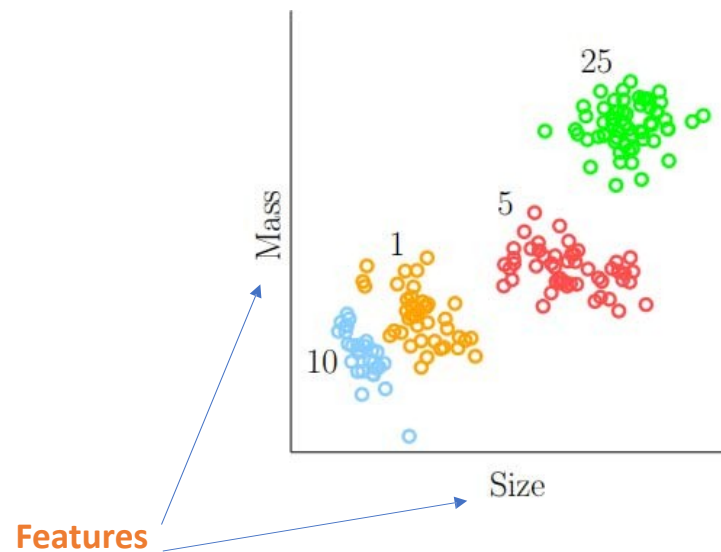
“A field of study that gives computers the ability to learn without being explicitly programmed.”



A machine-learning system is *trained* rather than explicitly programmed.

# Types of ML Systems

**Supervised Learning** - Training Data contains desired solutions, or *labels*



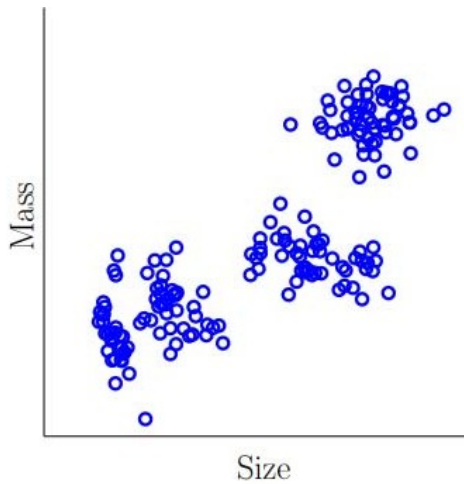
# Types of ML Systems

**Supervised Learning** - Training Data contains desired solutions, or *labels*

1. K-Nearest Neighbors
2. Linear Regression
3. Logistic Regression
4. Support Vector Machines
5. Decision Trees and Random Forests
6. Neural Networks

# Types of ML Systems

**Unsupervised Learning** - Training Data is *unlabeled*



# Types of ML Systems

**Unsupervised Learning** - Training Data is *unlabeled*

1. K-Means (Clustering)
2. Principal Component Analysis (Dimensionality Reduction)
3. Deep Neural Networks

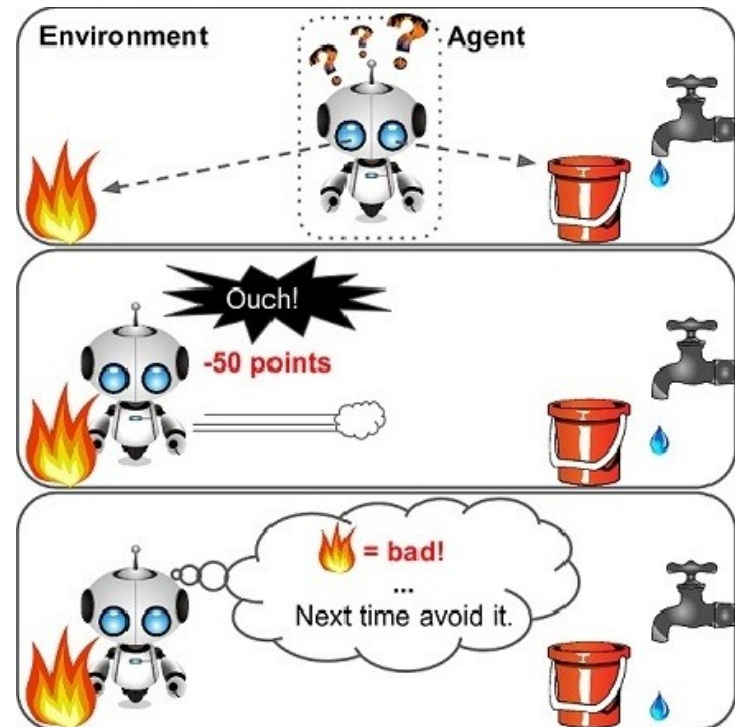
# Types of ML Systems

**Reinforcement Learning** - Training Data does not contain target output, but instead contains **some** possible output together with a measure of how **good** that output is.

<input>, <correct output>

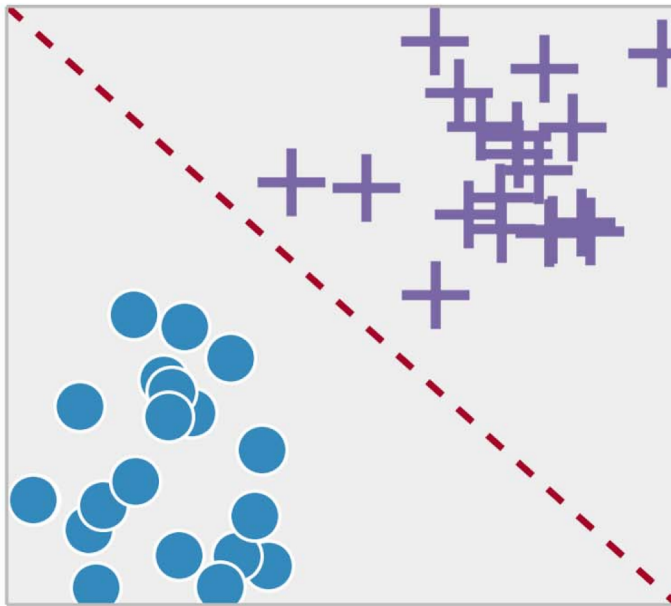


<input>, <some output>, <grade for this output>

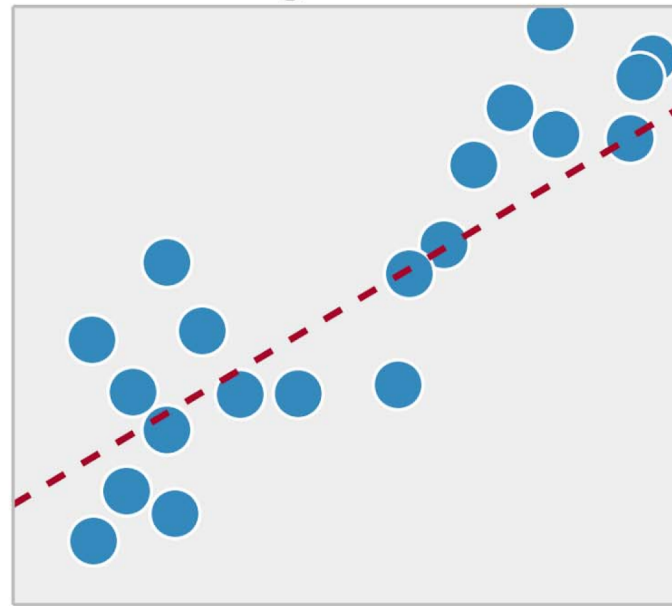


# Classification vs Regression

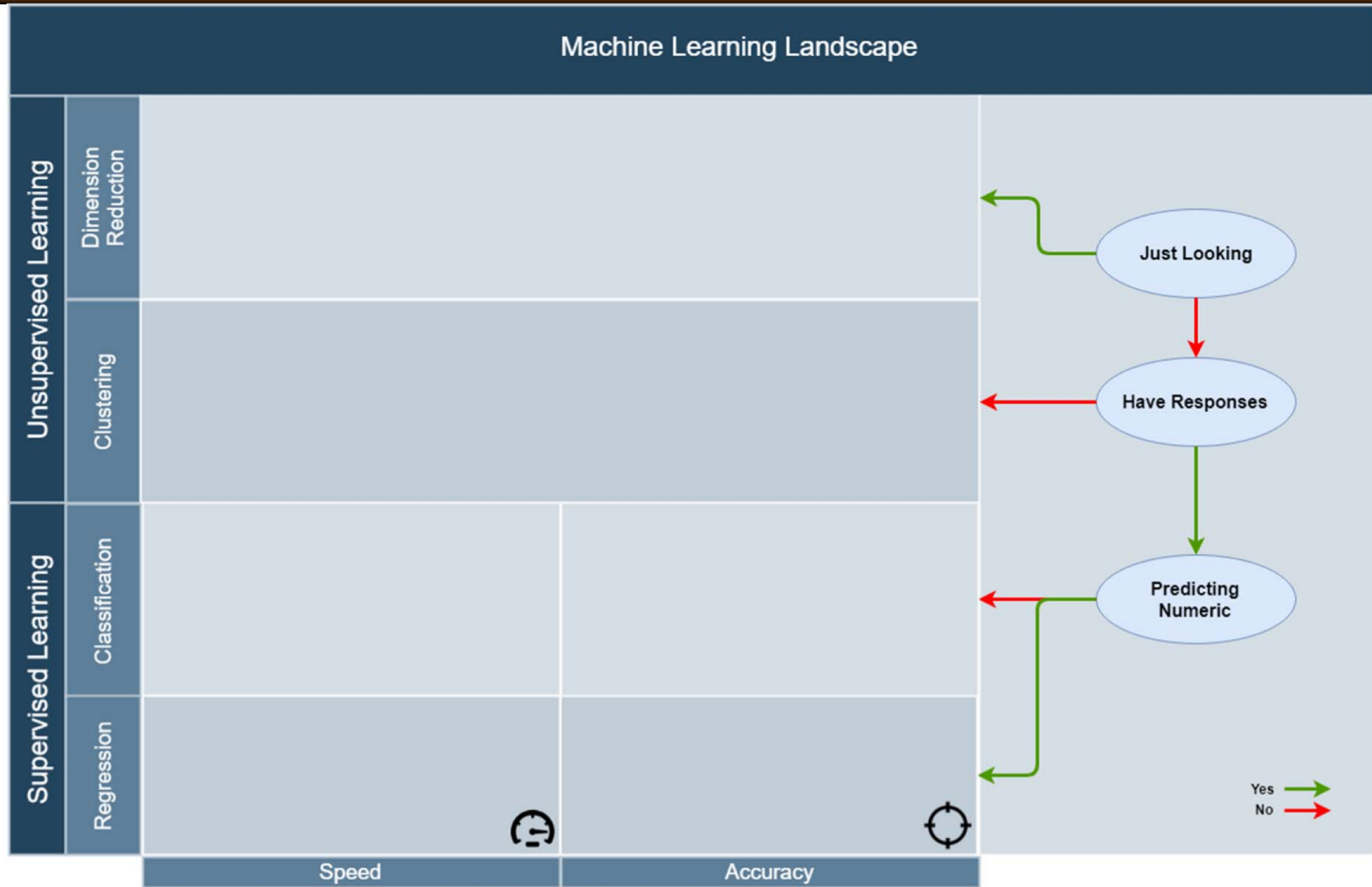
Classification



Regression



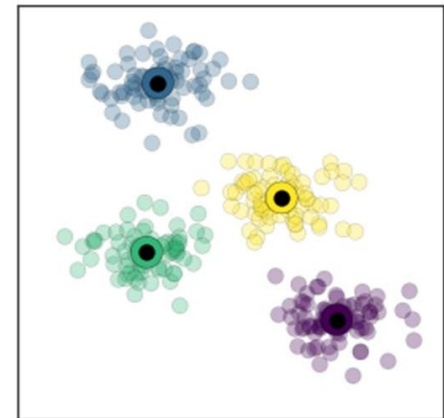
# ML Landscape



# Unsupervised Learning - Clustering

## Clustering

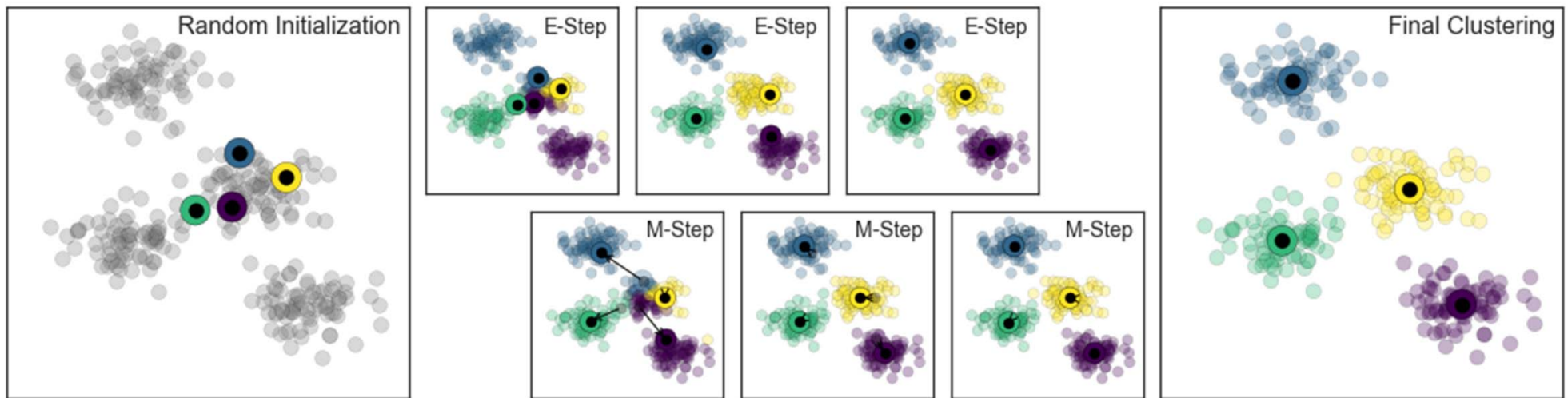
- Color “clusters” of points in a homogenous cloud of data.
- **Use Cases**
  - Behavioral Segmentation in Marketing
  - Useful as a pre-processing step before applying other classification algorithms.
    - Cluster ID could be added as feature for each data point.



# Unsupervised Learning - Clustering

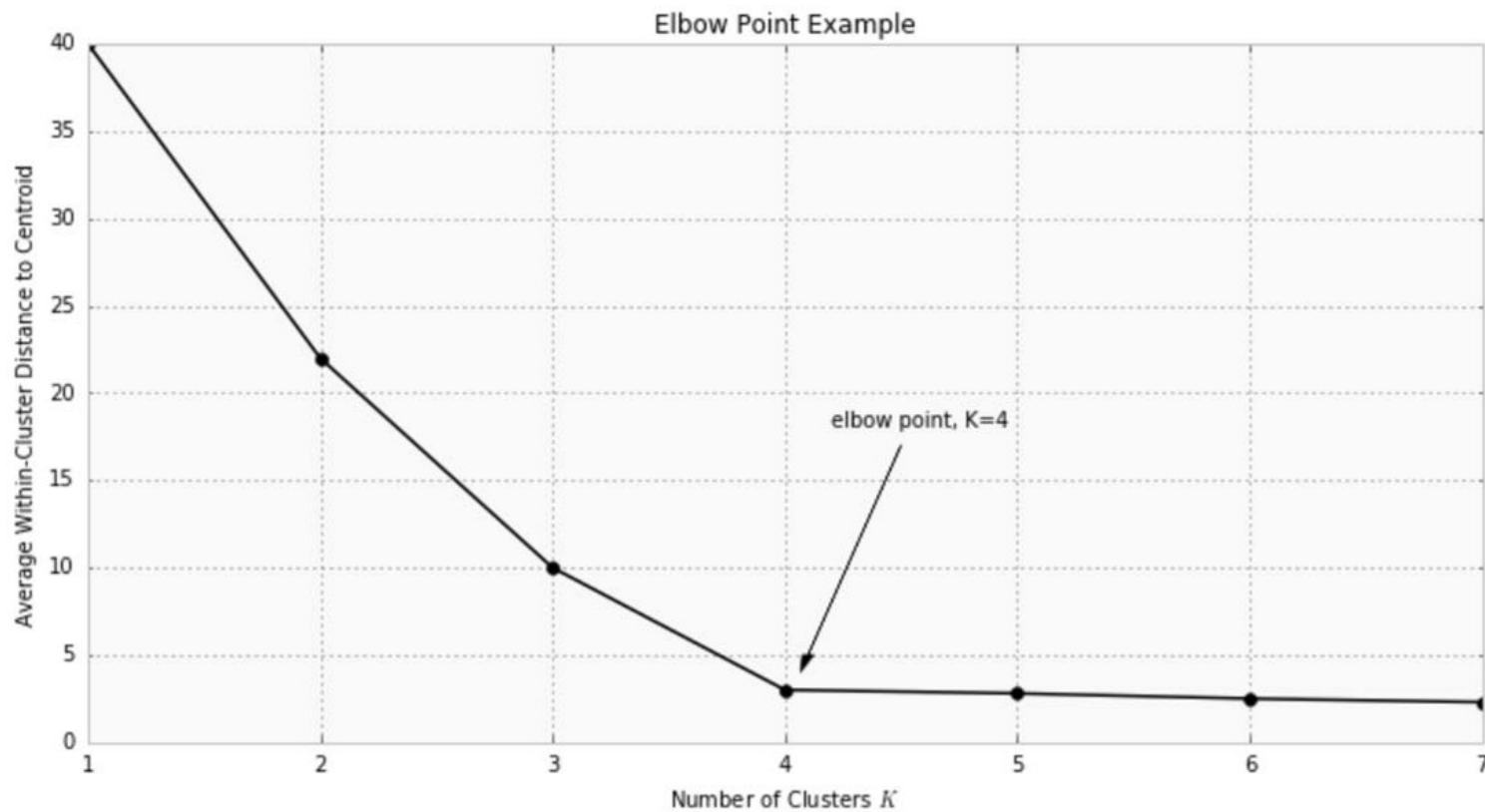
## k-means Algorithm

- Guess some cluster centers
- Repeat until converged
  - *E-Step*: assign points to the nearest cluster center
  - *M-Step*: set the cluster centers to the mean

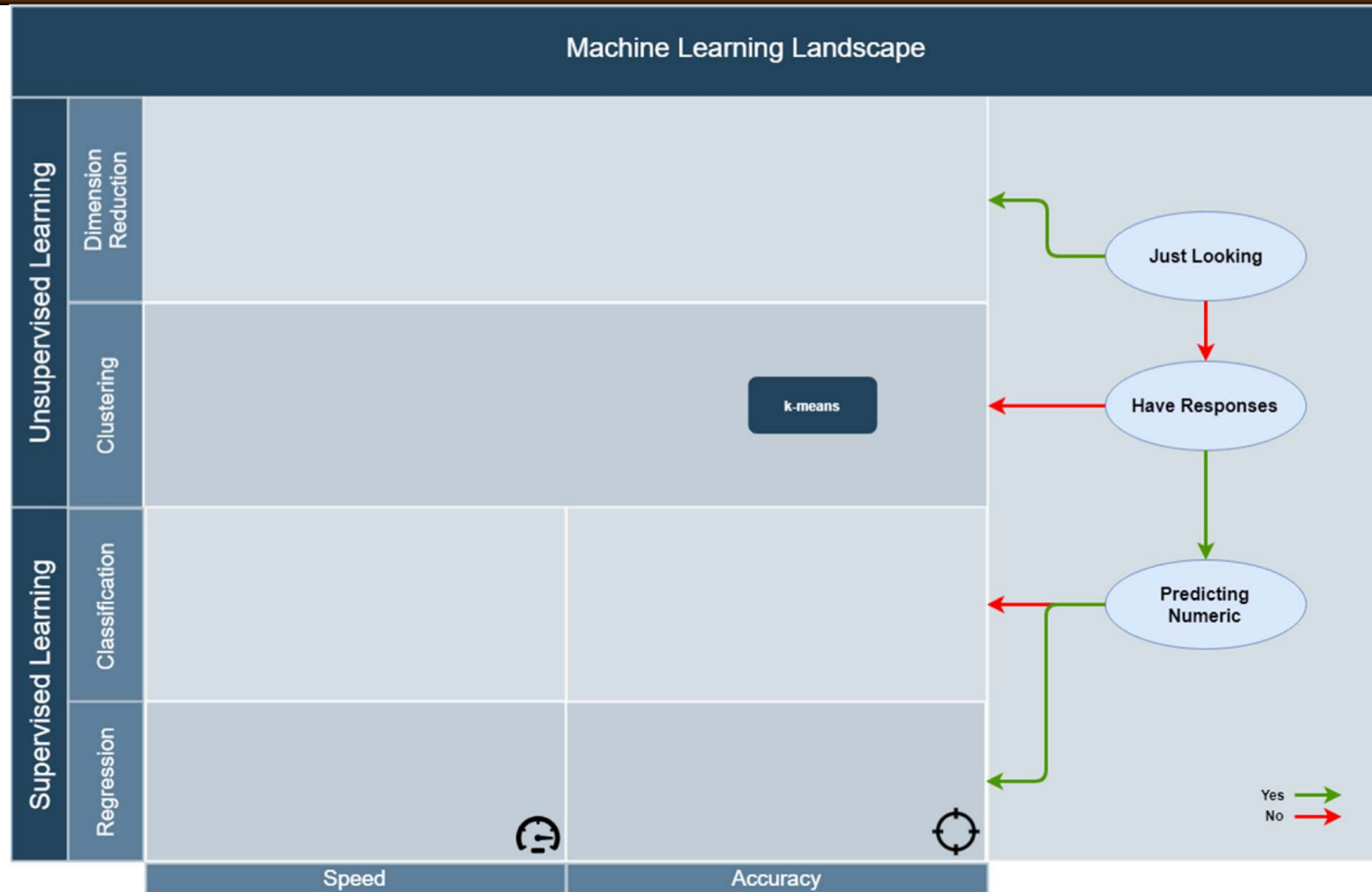


# Unsupervised Learning - Clustering

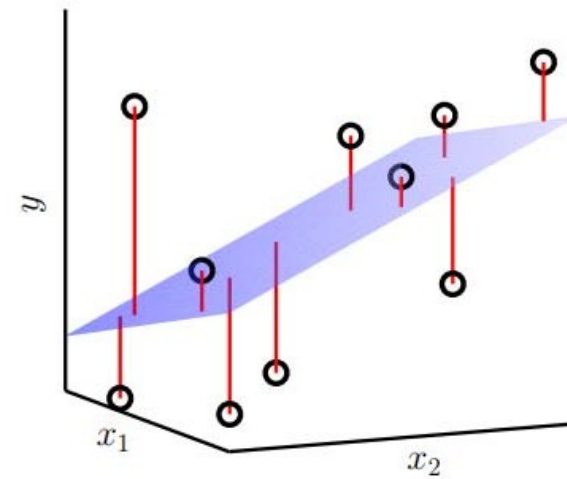
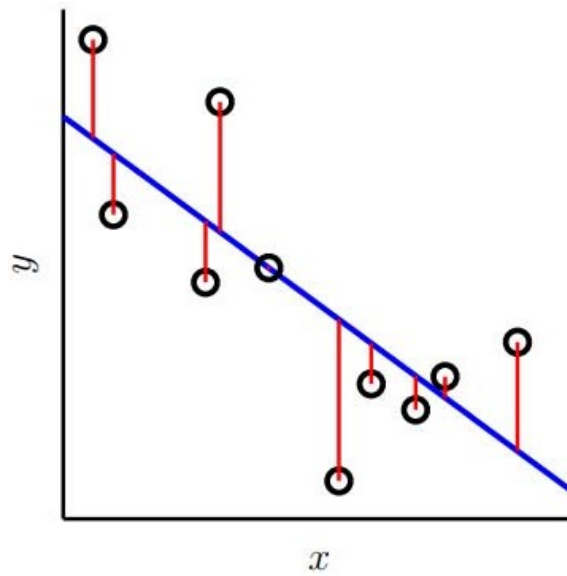
## Choosing $k$



# ML Landscape



# Linear Regression



# Linear Regression

$$X = \begin{bmatrix} 1 & x_1^1 & x_2^1 & \dots & x_n^1 \\ 1 & x_1^2 & x_2^2 & \dots & x_n^2 \\ \vdots & \vdots & \dots & \dots & \vdots \\ 1 & x_1^m & x_2^m & \dots & x_n^m \end{bmatrix}$$

$$y = \begin{bmatrix} y^1 \\ y^2 \\ \dots \\ y^m \end{bmatrix}$$

$$\Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \quad \hat{y} = \begin{bmatrix} y^1 \\ y^2 \\ \dots \\ y^m \end{bmatrix}$$

Define a **Hypothesis**

$$h_{\Theta}(X) = \hat{y} = X \Theta$$

Define a **Cost Function** (a measure of how bad we're doing)

$$MSE(X, h_{\Theta}) = \frac{1}{2} \sum_{i=1}^n [y^i - \hat{y}^i]^2$$

Repeat until convergence:

- Calculate Cost Function on chosen  $\Theta$
- Calculate slope of Cost Function
- Tweak  $\Theta$  so as to move downhill (reduce Cost Function value)

$\Theta$  is now optimized for our training data.

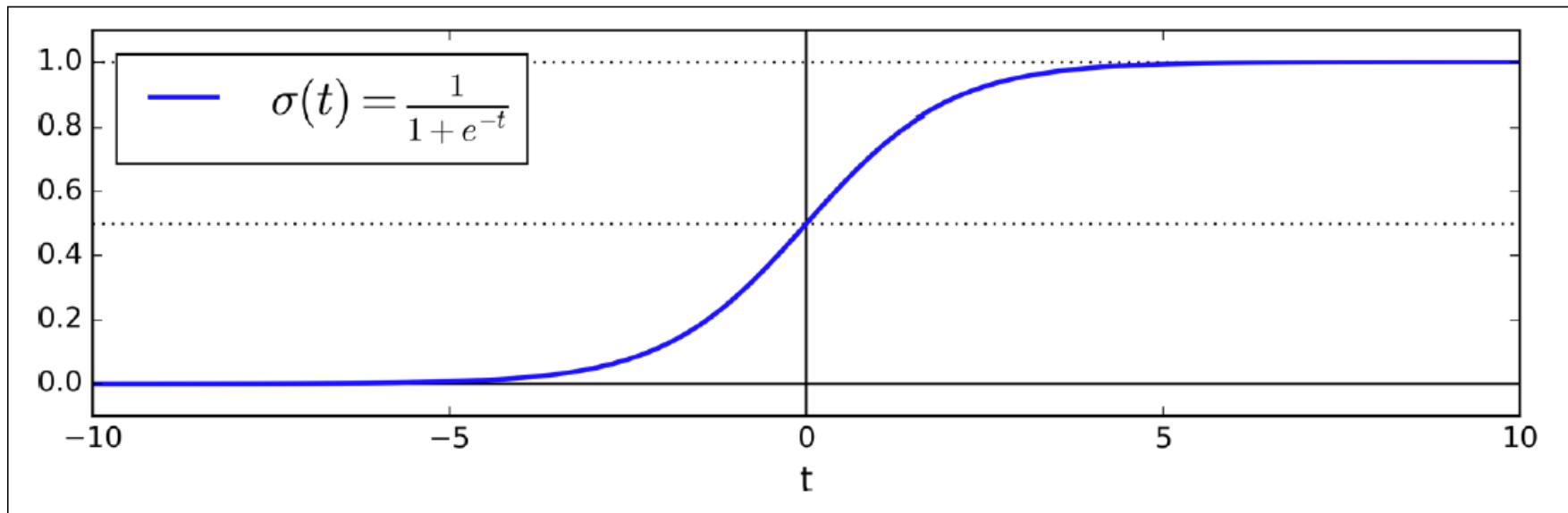
# Logistic Regression

Used to estimate the probability that an instance belongs to a particular class.

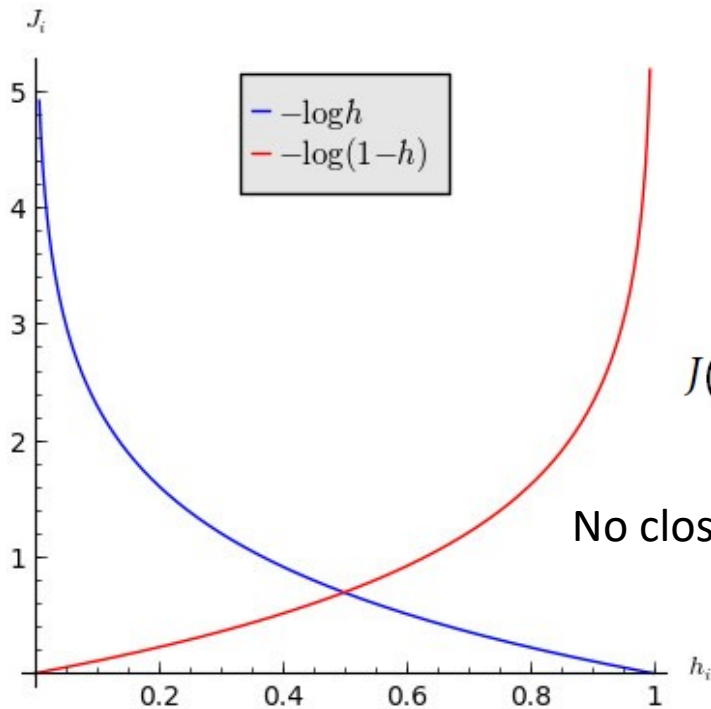
$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\theta^T \cdot \mathbf{x})$$

# Logistic Regression

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$



# Logistic Regression

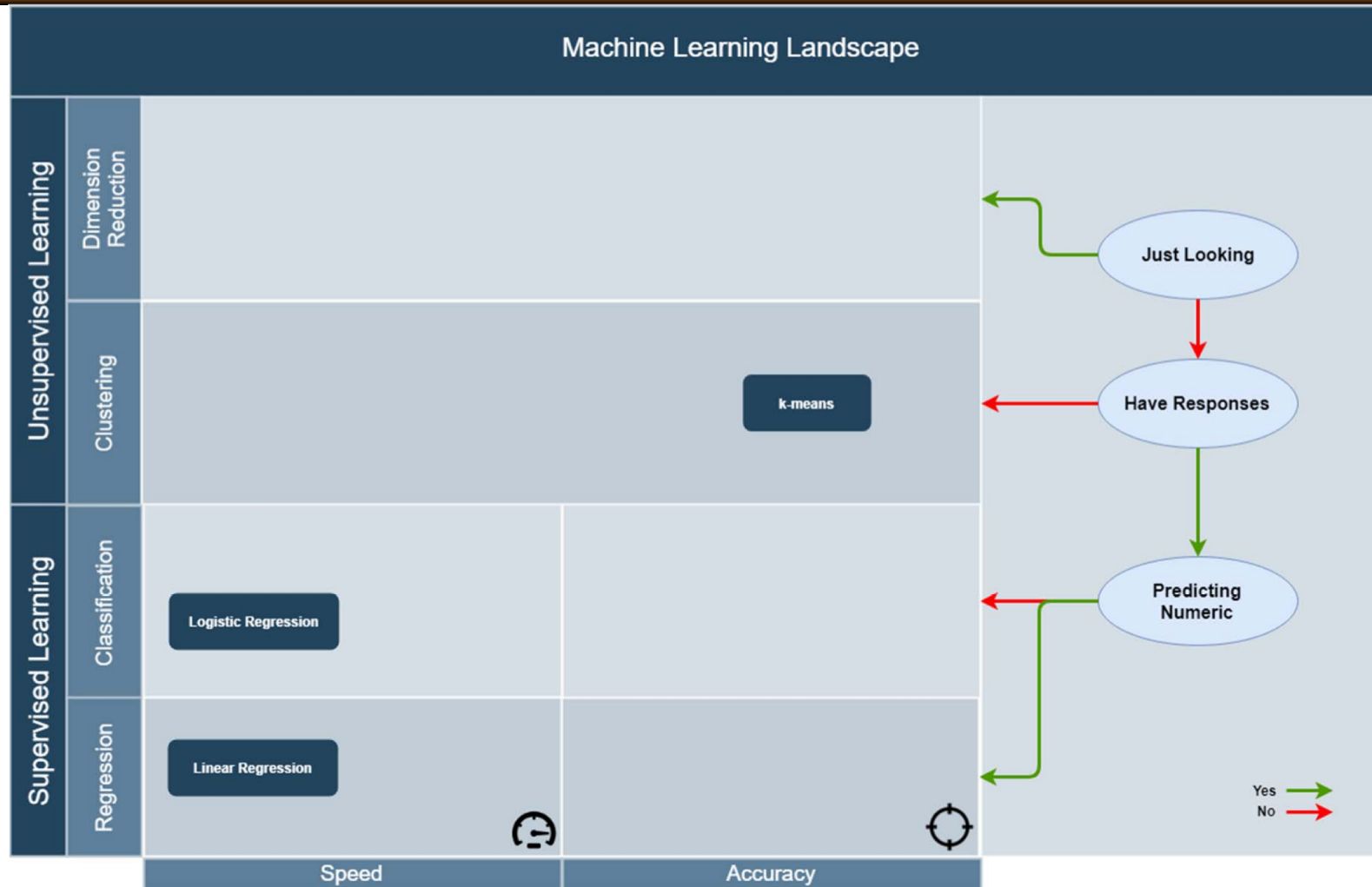


$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1, \\ -\log(1 - \hat{p}) & \text{if } y = 0. \end{cases}$$

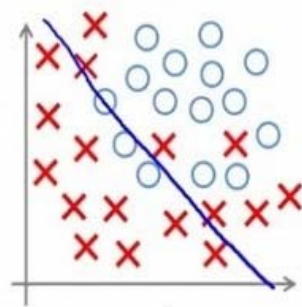
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

No closed-form solution, but we can use **Gradient Descent**!

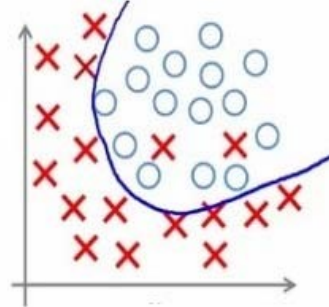
# ML Landscape



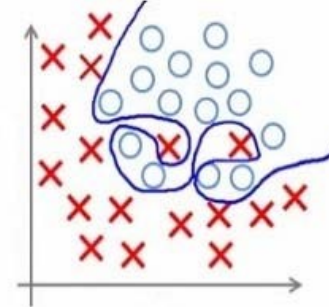
# Overfitting and Underfitting



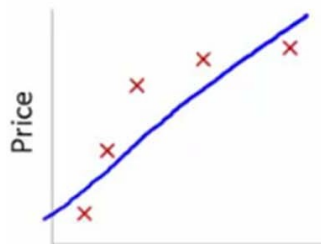
Under-fitting



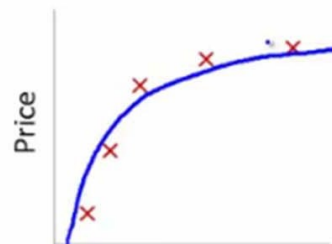
Appropriate-fitting



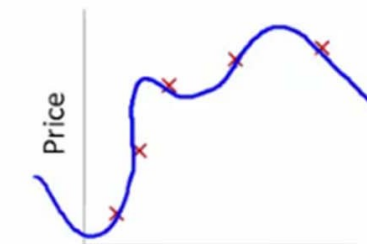
Over-fitting



Size  
 $\theta_0 + \theta_1 x$

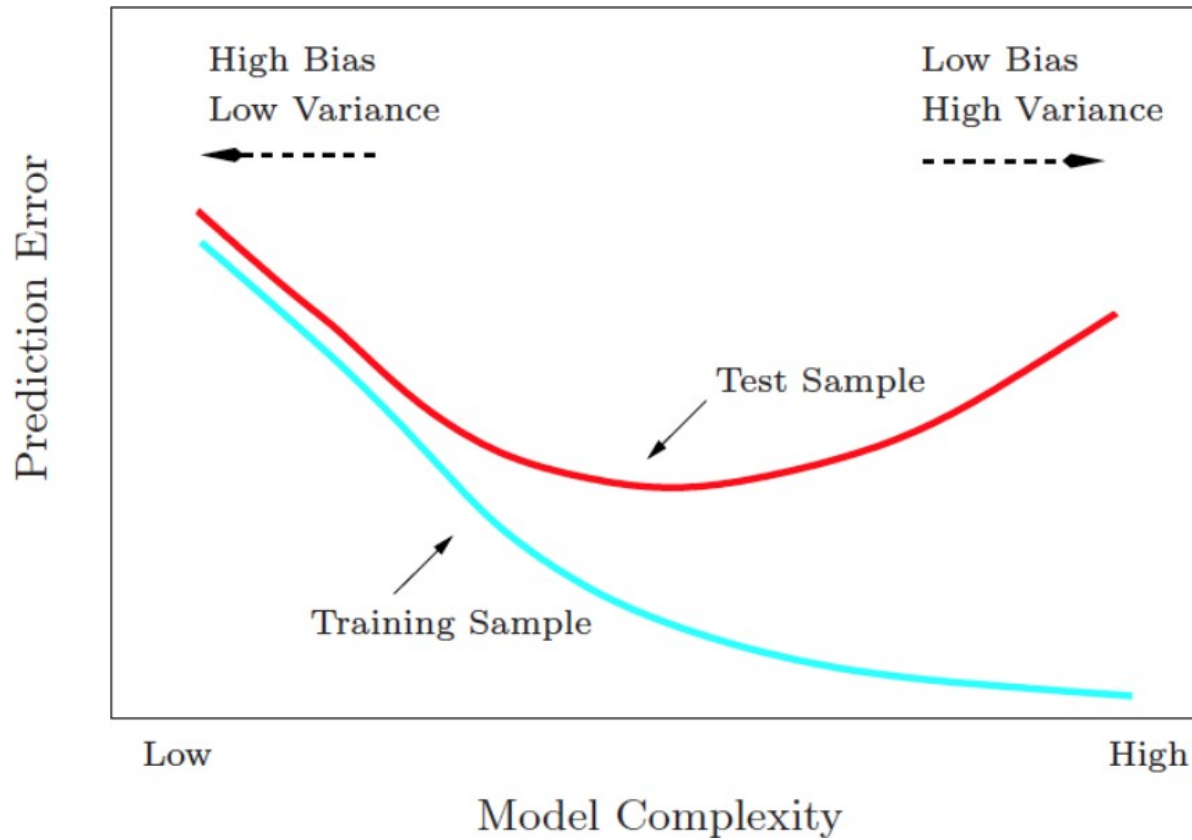


Size  
 $\theta_0 + \theta_1 x + \theta_2 x^2$



Size  
 $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

# Bias-Variance Tradeoff



# Regularization

How to ensure that we're not *overfitting* to our training data?

Impose a small penalty on model complexity.

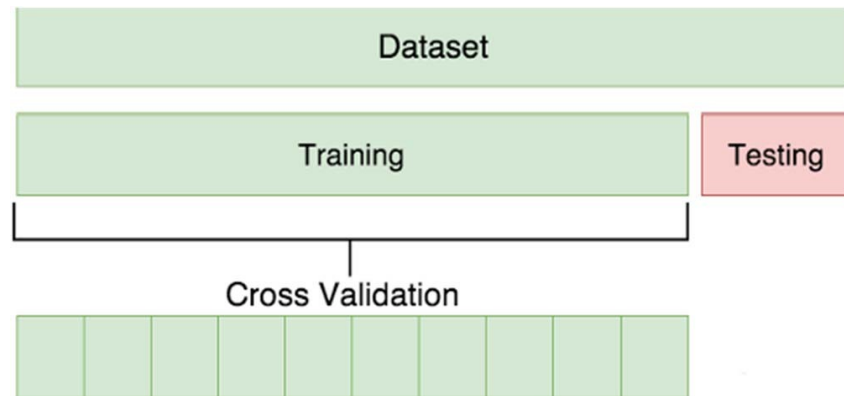
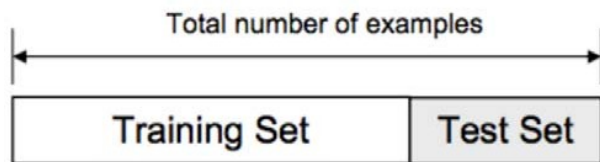
l1 penalty (Lasso Regression)

$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_{i=1}^n |\theta_i|$$

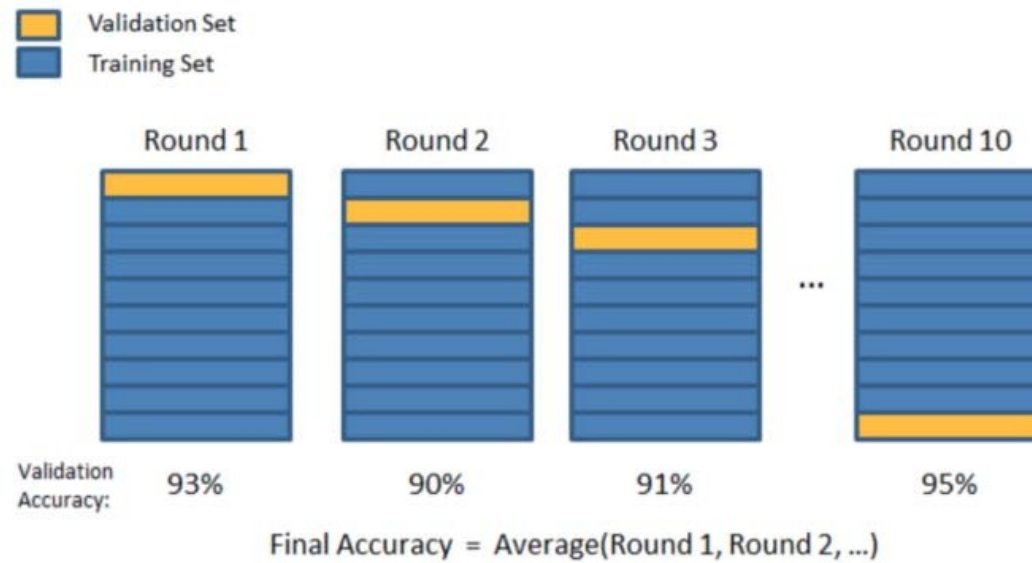
l2 penalty (Ridge Regression)

$$J(\theta) = \text{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

# Testing and Validation



# K-fold Cross Validation



# ML Landscape

scikit-learn  
algorithm cheat-sheet

